Problem when using ADAM with QAOA and the qasm_simulator

https://github.com/qiskit-community/qiskit-aqua/issues/1155

ROW 39

Problem when using ADAM with QAOA and the qasm_simulator #1155

Closed

EliasCombarro opened on Jul 31, 2020

Information

- **Qiskit Aqua version**: 0.7.3
- **Python version**: The one in the IBM Quantum Experience Jupyter notebook server
- **Operating system**: Program executed on the IBM Quantum Experience Jupyter notebook server

What is the current behavior?

When using QAOA with ADAM as the optimizer and setting a seed_simulator value, the optimizer seems to make only one function evaluation and return immediately. Also, even when not setting a seed, the results returned by the optimizer seem to be very bad. This seems to happen from Qiskit version 0.19.4 onwards. I have executed the same code with Qiskit 0.19.3 locally and there results are the expected (close to the ground state of the Hamiltonian) with and without seed. Also, this seems to be related only to the qasm_simulator. With the statevector_simulator, the results are as expected. Moreover, with other optimizers (for instance, COBYLA) results seem to be correct with both simulators.

Steps to reproduce the problem

You can use the following code:

```
from qiskit import Aer
from qiskit.aqua import aqua_globals, QuantumInstance
from qiskit.aqua.algorithms import QAOA
from qiskit.aqua.components.optimizers import ADAM
from qiskit.quantum_info import Pauli
from qiskit.aqua.operators import WeightedPauliOperator

pauli_list =[ [-1,Pauli([1],[0])] ]
q_op = WeightedPauliOperator(paulis = pauli_list)
rep = 10
backend = Aer.get_backend('qasm_simulator')
quantum_instance = QuantumInstance(backend, seed_simulator = 10)#, seed_transpiler = 1)

for i in range(rep):
optimizer = ADAM()
qaoa = QAOA(q_op, optimizer, p=1)
result = qaoa.run(quantum_instance)
#print(result)
print("***** Iteration ", i, "******")
print("Optimal value", result['optimal_value'])
print("Optimizer evals", result['optimizer_evals'])

quantum_instance = QuantumInstance(backend)

for i in range(rep):
optimizer = ADAM()
qaoa = QAOA(q_op, optimizer, p=1)
result = qaoa.run(quantum_instance)
#print(result)
print("***** Iteration ", i, "******")
print("Optimal value", result['optimal_value'])
print("Optimizer evals", result['optimizer_evals'])
```

The output is the following:

```
***** Iteration 0 *****
Optimal value 0.205078125
Optimizer evals 1
***** Iteration 1 *****
Optimal value 0.10546875
Optimizer evals 1
***** Iteration 2 *****
Optimal value 0.03125
Optimizer evals 1
***** Iteration 3 *****
```

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

***** Iteration 5 *****
Optimal value 0.02734375
Optimizer evals 1
***** Iteration 6 *****
Optimal value 0.04296875
Optimizer evals 1
***** Iteration 7 *****
Optimal value 0.380859375
Optimizer evals 1
***** Iteration 8 *****
Optimal value 0.02734375
Optimizer evals 1
***** Iteration 9 *****
Optimal value 0.04296875
Optimizer evals 1

***** Iteration 0 *****
Optimal value 0.171875
Optimizer evals 10000
***** Iteration 1 *****
Optimal value 0.14453125
Optimizer evals 10000
***** Iteration 2 *****
Optimal value 0.05859375
Optimizer evals 6408
***** Iteration 3 *****
Optimal value 0.1171875
Optimizer evals 10000
***** Iteration 4 *****
Optimal value 0.150390625
Optimizer evals 10000
***** Iteration 5 *****
Optimal value 0.10546875
Optimizer evals 10000
***** Iteration 6 *****
Optimal value 0.587890625
Optimizer evals 10000
***** Iteration 7 *****
Optimal value 0.041015625
Optimizer evals 10000

The problem is twofold: on the one hand, when using a seed the optimizer seems to stop prematurely; on the other, even without the seed, the results are far from the optimum solution of the problem.

What is the expected behavior?

This is the output of the same code when run with Qiskit 0.19.3 (Aqua 0.7.1)

***** Iteration 0 *****
Optimal value -0.99999999997112
Optimizer evals 1927
***** Iteration 1 *****
Optimal value -0.99999999807446
Optimizer evals 2711
***** Iteration 2 *****
Optimal value -0.99999999995963
Optimizer evals 2729
***** Iteration 3 *****
Optimal value -0.99999999995898
Optimizer evals 2720
***** Iteration 4 *****
Optimal value -0.99999999987783
Optimizer evals 2626
***** Iteration 5 *****
Optimal value -0.9999999998064
Optimizer evals 2287
***** Iteration 6 *****
Optimal value -0.99999999999565
Optimizer evals 2649

***** Iteration 3 *****
Optimal value -0.99999999980308
Optimizer evals 2080
***** Iteration 4 *****
Optimal value -0.99999999998902
Optimizer evals 2372
***** Iteration 5 *****
Optimal value -0.99999999971857
Optimizer evals 2466
***** Iteration 6 *****
Optimal value -0.99999999986107
Optimizer evals 2378
***** Iteration 7 *****
Optimal value -0.99999999988471
Optimizer evals 2050
***** Iteration 8 *****
Optimal value -0.99999999998525
Optimizer evals 2005
***** Iteration 9 *****
Optimal value -0.99999999687778
Optimizer evals 2324

And this is the output in Qiskit 0.19.6 (Aqua 0.7.3) but changing the qasm_simulator to the statevector_simulator:

***** Iteration 0 *****
Optimal value -0.99999999999926
Optimizer evals 2292
***** Iteration 1 *****
Optimal value -0.99999999996109
Optimizer evals 2310
***** Iteration 2 *****
Optimal value -0.99999999996412
Optimizer evals 2845
***** Iteration 3 *****
Optimal value -0.99999999999846
Optimizer evals 2221
***** Iteration 4 *****
Optimal value -0.99999999999081
Optimizer evals 2239
***** Iteration 5 *****
Optimal value -0.99999999997348
Optimizer evals 2262
***** Iteration 6 *****
Optimal value -0.99999999533563
Optimizer evals 2174
***** Iteration 7 *****
Optimal value -0.99999999992073
Optimizer evals 2770
***** Iteration 8 *****
Optimal value -0.99999998954988
Optimizer evals 10000
***** Iteration 9 *****
Optimal value -0.99999999999545
Optimizer evals 1968

***** Iteration 0 *****
Optimal value -0.99999999998019
Optimizer evals 2357
***** Iteration 1 *****
Optimal value -0.99999999612502
Optimizer evals 2135
***** Iteration 2 *****
Optimal value -0.99999999988254
Optimizer evals 2052
***** Iteration 3 *****
Optimal value -0.99999999997709
Optimizer evals 2775
***** Iteration 4 *****
Optimal value -0.99999999875184

Optimizer evals 2349
***** Iteration 7 *****
Optimal value -0.99999999991085
Optimizer evals 2313
***** Iteration 8 *****
Optimal value -0.99999999985751
Optimizer evals 1993
***** Iteration 9 *****
Optimal value -0.99999999782576
Optimizer evals 2085

Suggested solutions

The problem seems to be in the interaction between ADAM and the qasm_simulator (and, probably, the seed of the simulator) and it may be caused by some change made from Qiskit 0.19.3 to 0.19.4

woodsp-ibm on Aug 1, 2020 — Member

QAOA directly extends the VQE algorithm in our implementation. I say this as the expectation value computed is done so via logic in VQE which uses an `expectation` object and the default expectation computation, which is selected when None is explicitly provided by the user, was changed when using Aer and qasm_simulator in 0.19.4. This user provided expectation capability was new function that was added in 0.19 (Aqua 0.7) - it was more internal in earlier releases. Now the default expectation chosen when using Aer qasm_simulator was initially picked for best performance - this is an AerPauilExpectation - which uses a specific capability of Aer, that allows it to more directly do the expectation computation via a snapshot instruction. The net behavior in this 'mode' is however like the outcome of the statevector simulator in that it is ideal and does not have shot noise due to sampling despite using the qasm simulator. Prior to Aqua 0.7 one could select this 'mode' of operation with the Aer simulator but it was not the default. After releasing this in 0.19 we got feedback from users saying they would pick qasm simulator since they wanted the more device-like behavior with shot noise and it no longer worked like it did in the past. So based on user feedback we changed the default over to the "regular" PauliExpectation that does not use that snapshot and now has shot noise once again. You can read more via the change #1040. I imagine this shot noise is causing the change you see.

So bottom line here is that if you add `include_custom=True` when you create QAOA see https://qiskit.org/documentation/stubs/qiskit.aqua.algorithms.QAOA.html for more info, it will go back to include the AerPauliExpectation, that uses the custom Aer instruction such that this will be selected when none is given once again - so the net behavior will be once again like you saw it. Hopefully this explains what you are seeing and changing your QAOA to `qaoa = QAOA(q_op, optimizer, p=1, include_custom=True)` brings you back to where you were. (You could explicitly provide an instance of that expectation if you prefer `qaoa = QAOA(q_op, optimizer, p=1, expectation=AerPauliExpectation)` given the you import that). Let us know whether this sorts things for you, as I believe it should.

EliasCombarro on Aug 3, 2020 — Author

Thank you very much. I have tried with the "include_custom = True" option and it seems to work now.

woodsp-ibm on Aug 3, 2020 — Member

Great, I am closing this issue off then since the solution provided is working for you.

woodsp-ibm closed this as completed on Aug 3, 2020